🚀 CATEGORY 10: Advanced Patterns, Testing & Troubleshooting

> **Difficulty:** Advanced | **Topics:** 15 | **Terraform Interview Mastery Series**

Table of Contents

1. ⚠ Multi-Account / Multi-Region Patterns
2. Multi-Cloud Patterns — Provider Aliases, Shared Modules
3. Terragrunt — What It Solves, When to Use Over Vanilla Terraform
4. ⚠ Large-Scale Terraform — Monorepo vs Polyrepo, State Splitting
5. ➕ ⚠ `terraform graph` — DAG Internals, Cycle Errors Deep Dive
6. Testing — `terraform validate`, `fmt`, Native `terraform test` (1.6+)
7. ⚠ Terratest — Go-Based Integration Testing for Modules
8. ➕ ⚠ Mocking Providers in Tests — Patterns and Frameworks
9. ➕ Contract Testing Between Modules — Interface/API Design
10. ➕ Testing Module Contracts & Output Validation
11. `TF_LOG` Levels — Debugging Provider and Core Issues
12. ⚠ State Corruption — Causes, Recovery, Prevention
13. `terraform force-unlock` Advanced Scenarios
14. ➕ Provider Plugin Crash Debugging — Reading Crash Logs, Reporting
15. ➕ ⚠ Cycle Errors — Deep Internal Understanding, How DAG Detects Them

---

Topic 73: ⚠ Multi-Account / Multi-Region Patterns

🔵 What It Is (Simple Terms)

Production infrastructure spans multiple AWS accounts (for isolation, blast radius, billing) and multiple regions (for latency, DR, compliance). Terraform must manage resources across these boundaries from a single configuration.

🔵 Why Multi-Account Architecture

```
AWS Organizations best practice:
  management-account  ← billing, consolidated, NO resources
  security-account    ← GuardDuty, SecurityHub, CloudTrail aggregation
  log-archive-account ← centralized CloudTrail, VPC flow logs
  shared-services     ← shared Route53, Transit Gateway, ECR
  prod-account        ← production workloads
  staging-account     ← staging workloads
  dev-account         ← dev workloads (sandbox)
```

```
    sandbox-account     ← individual developer experiments

  Benefits:
    → Blast radius: prod bug can't affect other accounts
    → Billing: clear per-team cost allocation
    → Security: prod IAM is completely isolated from dev
    → Compliance: prod environment boundary = account boundary
    → Limit exposure: even admin access to dev can't touch prod
```

🔵 Pattern 1: Provider Aliases for Multi-Account

```
# root module: providers.tf

# Default provider — prod account (assumed via OIDC/role chain)
provider "aws" {
  region = "eu-west-1"
  # Uses current execution credentials (CI/CD OIDC role in prod account)
}

# Shared services account — assume role cross-account
provider "aws" {
  alias  = "shared_services"
  region = "eu-west-1"

  assume_role {
    role_arn     = "arn:aws:iam::111122223333:role/TerraformCrossAccount"
    session_name = "terraform-shared-services"
    external_id  = var.external_id   # prevent confused deputy attack
  }
}

# Security account — centralized logging
provider "aws" {
  alias  = "security"
  region = "eu-west-1"

  assume_role {
    role_arn     = "arn:aws:iam::444455556666:role/TerraformCrossAccount"
    session_name = "terraform-security"
  }
}

# Log archive account
provider "aws" {
  alias  = "log_archive"
  region = "eu-west-1"

  assume_role {
    role_arn = "arn:aws:iam::777788889999:role/TerraformCrossAccount"
    session_name = "terraform-log-archive"
```

```
  }
}
```

```
# Using cross-account providers in resources
resource "aws_vpc" "prod" {
  cidr_block = "10.0.0.0/16"
  # Uses default provider (prod account)
}

resource "aws_route53_zone" "internal" {
  provider = aws.shared_services   # creates zone in shared services
account
  name     = "internal.mycompany.com"
  vpc {
    vpc_id = aws_vpc.prod.id   # VPC in prod account
  }
}

resource "aws_s3_bucket" "audit_logs" {
  provider = aws.log_archive     # creates bucket in log archive account
  bucket   = "mycompany-prod-audit-logs"
}
```

🔵 Pattern 2: Multi-Region with Aliases

```
# providers.tf — multi-region for DR / ACM requirements

provider "aws" {
  region = "eu-west-1"    # primary region
}

provider "aws" {
  alias  = "us_east_1"
  region = "us-east-1"    # ACM certificates for CloudFront MUST be in us-
east-1
}

provider "aws" {
  alias  = "eu_central_1"
  region = "eu-central-1" # DR region
}

# ACM cert in us-east-1 for CloudFront
resource "aws_acm_certificate" "cdn" {
  provider          = aws.us_east_1   # CloudFront requirement
  domain_name       = "myapp.com"
  validation_method = "DNS"
}
```

```
# CloudFront uses cert from us-east-1
resource "aws_cloudfront_distribution" "main" {
  # default provider (eu-west-1) — CloudFront is global anyway
  viewer_certificate {
    acm_certificate_arn = aws_acm_certificate.cdn.arn   # cross-region
reference
  }
}

# DR replication bucket
resource "aws_s3_bucket" "dr" {
  provider = aws.eu_central_1
  bucket   = "mycompany-prod-dr-data"
}

resource "aws_s3_bucket_replication_configuration" "main" {
  # primary bucket (eu-west-1) → DR bucket (eu-central-1)
  bucket = aws_s3_bucket.primary.id
  role   = aws_iam_role.replication.arn

  rule {
    destination {
      bucket = aws_s3_bucket.dr.arn   # cross-region reference
    }
  }
}
```

🔵 Pattern 3: Passing Providers to Modules

```
# modules/vpc/versions.tf — declare provider requirements
terraform {
  required_providers {
    aws = {
      source                = "hashicorp/aws"
      version               = ">= 4.0"
      configuration_aliases = [aws.primary, aws.secondary]  # declare
aliases module uses
    }
  }
}

# modules/vpc/main.tf — use aliased providers
resource "aws_vpc" "primary" {
  provider   = aws.primary
  cidr_block = var.primary_cidr
}

resource "aws_vpc" "secondary" {
  provider   = aws.secondary
```

```
  cidr_block = var.secondary_cidr
}
```

```
# root module — pass providers to module
module "networking" {
  source = "./modules/vpc"

  primary_cidr   = "10.0.0.0/16"
  secondary_cidr = "10.1.0.0/16"

  providers = {
    aws.primary   = aws              # root's default provider
    aws.secondary = aws.eu_central_1 # root's aliased provider
  }
}
```

🔵 Cross-Account IAM Trust — Required Setup

```
# In each target account: allow management account to assume this role
resource "aws_iam_role" "terraform_cross_account" {
  name = "TerraformCrossAccount"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect    = "Allow"
      Principal = {
        AWS =
"arn:aws:iam::${var.management_account_id}:role/TerraformRole-Prod"
      }
      Action    = "sts:AssumeRole"
      Condition = {
        StringEquals = {
          "sts:ExternalId" = var.external_id   # mitigates confused deputy
        }
      }
    }]
  })
}

# Attach appropriate permissions to TerraformCrossAccount in each account
resource "aws_iam_role_policy_attachment" "cross_account_permissions" {
  role       = aws_iam_role.terraform_cross_account.name
  policy_arn = aws_iam_policy.terraform_permissions.arn
}
```

🔵 Short Interview Answer

> "Multi-account Terraform uses provider aliases with `assume_role` blocks — each provider alias targets a different AWS account by assuming a cross-account IAM role. The execution role in the primary account assumes roles in target accounts to make API calls. Cross-account roles need a trust policy allowing the primary Terraform role to assume them, with an `external_id` to prevent the confused deputy attack. For multi-region, you add aliases with different `region` values — common for ACM certificates (must be in us-east-1 for CloudFront), DR configurations, and cross-region replication. Modules that use aliased providers must declare `configuration_aliases` in their `required_providers` block and receive the aliases via the `providers` map when called."

Topic 74: Multi-Cloud Patterns — Provider Aliases, Shared Modules

---

🔵 What It Is (Simple Terms)

Multi-cloud means managing resources across multiple cloud providers (AWS + GCP + Azure) from a single or coordinated set of Terraform configurations. Common for organizations with multi-cloud strategies, acquired companies, or when specific services are best-of-breed on each cloud.

🔵 When Multi-Cloud Terraform Makes Sense

```
Legitimate use cases:
  ✅ AWS primary + Cloudflare DNS/WAF (very common)
  ✅ AWS primary + GCP BigQuery for data analytics
  ✅ Azure AD + AWS resources (enterprise SSO requirements)
  ✅ Post-acquisition: company A on AWS, company B on GCP
  ✅ PagerDuty/Datadog/Snowflake resources managed alongside cloud infra

Problematic use cases (avoid):
  ❌ "Multi-cloud for resilience" — abstractions leak, tooling complexity
high
  ❌ Same app on AWS and GCP simultaneously — networking, latency, cost
complexity
  ❌ Multi-cloud as vendor lock-in avoidance — modules abstract only so
much
```

🔵 Practical Multi-Cloud Configuration

```
# providers.tf — multiple cloud providers

provider "aws" {
```

```
  region = "eu-west-1"
}

provider "google" {
  project = var.gcp_project_id
  region  = "europe-west1"
}

provider "azurerm" {
  features {}
  subscription_id = var.azure_subscription_id
}

provider "cloudflare" {
  api_token = var.cloudflare_api_token
}

provider "datadog" {
  api_key = var.datadog_api_key
  app_key = var.datadog_app_key
}

# Cross-provider resources referencing each other
resource "aws_lb" "app" {
  name               = "myapp-prod"
  load_balancer_type = "application"
}

resource "cloudflare_record" "app" {
  zone_id = var.cloudflare_zone_id
  name    = "app"
  value   = aws_lb.app.dns_name    # AWS resource → Cloudflare DNS record
  type    = "CNAME"
  proxied = true
}

resource "datadog_monitor" "app_availability" {
  name    = "App availability check"
  type    = "service check"
  message = "App is down! @pagerduty-prod"
  query   =
"\"http.can_connect\".over(\"url:https://app.mycompany.com\").last(3).coun
t_by_status()"
  # Uses Datadog provider to create monitoring for AWS-hosted app
}
```

🔵 Shared Modules Across Cloud Providers

```
# Modules can abstract away cloud-specific implementations
```

```
# interface: modules/object-storage/variables.tf
variable "name"        { type = string }
variable "environment" { type = string }
variable "versioning"  { type = bool; default = true }

# AWS implementation: modules/object-storage/aws/main.tf
resource "aws_s3_bucket" "main" {
  bucket = "${var.name}-${var.environment}"
}

resource "aws_s3_bucket_versioning" "main" {
  bucket = aws_s3_bucket.main.id
  versioning_configuration {
    status = var.versioning ? "Enabled" : "Suspended"
  }
}

# GCP implementation: modules/object-storage/gcp/main.tf
resource "google_storage_bucket" "main" {
  name     = "${var.name}-${var.environment}"
  location = "EU"
  versioning { enabled = var.versioning }
}

# Root module chooses implementation
module "app_storage" {
  source = var.cloud_provider == "aws" ? "./modules/object-storage/aws" :
"./modules/object-storage/gcp"

  name        = "app-data"
  environment = var.environment
  versioning  = true
}

# Limitation: outputs differ between implementations — abstraction leaks
# AWS outputs: bucket_arn, bucket_name, bucket_domain_name
# GCP outputs: bucket_url, bucket_name
# Callers must handle provider-specific output differences
```

🔵 Short Interview Answer

> "Multi-cloud Terraform is most practical when managing SaaS provider resources alongside cloud infrastructure — Cloudflare, Datadog, PagerDuty, Snowflake alongside AWS resources. You configure multiple provider blocks in the same root module and reference resources across them. Cross-provider references work naturally — a Cloudflare DNS record can reference an AWS ALB's DNS name. Shared module abstraction across cloud providers works for simple resources but leaks at the interface — AWS S3 and GCP Storage have different output shapes. For genuine multi-cloud workloads, the practical recommendation is separate Terraform configs per cloud with cross-cloud references through DNS, shared secrets in Vault, or API endpoints rather than direct state references."

Topic 75: Terragrunt — What It Solves, When to Use Over Vanilla Terraform

🔵 What It Is (Simple Terms)

Terragrunt is a thin wrapper around Terraform that adds features Terraform lacks natively: DRY backend configuration across many root modules, automatic dependency ordering between stacks, and before/after hooks. It's built by Gruntwork and widely used in large-scale Terraform deployments.

🔵 The Problems Terragrunt Solves

```
Problem 1: Backend config duplication
  You have 30 Terraform root modules (stacks)
  Each needs the same S3 backend config
  Change the bucket name → update 30 files

Problem 2: Running commands across stacks
  You want to apply networking, then security, then compute in order
  Vanilla Terraform: manually run apply in each directory in the right
order
  One directory fails → manual intervention required

Problem 3: DRY variable values
  Every stack needs the same environment, region, account_id
  Copy-pasting across 30 terraform.tfvars files
  One change needed → update 30 files

Problem 4: Before/after hooks
  Run `aws s3 sync` before apply to download configs
  Run tests after apply to validate
  Vanilla Terraform: no built-in hook mechanism

Terragrunt solves all of these:
  ✅ Backend config inheritance from root terragrunt.hcl
  ✅ run-all commands across directories with dependency ordering
  ✅ Inputs inheritance through directory hierarchy
  ✅ before_hook, after_hook, error_hook
```

🔵 Terragrunt Structure

```
infrastructure/
├── terragrunt.hcl              ← root config (shared backend, inputs)
├── prod/
```

```
│   ├── account.hcl             ← prod account config
│   ├── networking/
│   │   └── terragrunt.hcl      ← stack config
│   ├── security/
│   │   └── terragrunt.hcl
│   └── compute/
│       └── terragrunt.hcl
└── staging/
    ├── account.hcl
    ├── networking/
    │   └── terragrunt.hcl
    └── compute/
        └── terragrunt.hcl
```

```
# Root terragrunt.hcl — shared backend configuration
locals {
  # Read account config from the directory hierarchy
  account_vars =
read_terragrunt_config(find_in_parent_folders("account.hcl"))
  region       = "eu-west-1"
  account_id   = local.account_vars.locals.account_id
  environment  = local.account_vars.locals.environment
}

generate "backend" {
  path      = "backend.tf"   # generates this file in each module
directory
  if_exists = "overwrite_terragrunt"

  contents = <<EOF
terraform {
  backend "s3" {
    bucket         = "mycompany-terraform-state-${local.account_id}"
    key            = "${path_relative_to_include()}/terraform.tfstate"
    region         = "${local.region}"
    encrypt        = true
    kms_key_id     = "alias/terraform-state"
    dynamodb_table = "terraform-state-locks"
  }
}
EOF
}

generate "provider" {
  path      = "provider.tf"
  if_exists = "overwrite_terragrunt"

  contents = <<EOF
provider "aws" {
  region = "${local.region}"
  default_tags {
```

```
    tags = {
      Environment = "${local.environment}"
      ManagedBy   = "terraform"
      Account     = "${local.account_id}"
    }
  }
}
EOF
}

# Global inputs available to all stacks
inputs = {
  environment = local.environment
  account_id  = local.account_id
  region      = local.region
}
```

```
# prod/account.hcl
locals {
  account_id  = "123456789012"
  environment = "prod"
}
```

```
# prod/compute/terragrunt.hcl
include "root" {
  path = find_in_parent_folders()   # inherits root terragrunt.hcl
}

terraform {
  source = "../../modules/compute"   # points to module
}

# Declare dependency on networking stack
dependency "networking" {
  config_path = "../networking"
  mock_outputs = {                  # used during plan before networking
is applied
    vpc_id             = "vpc-mock-123"
    private_subnet_ids = ["subnet-mock-1", "subnet-mock-2"]
  }
  mock_outputs_allowed_terraform_commands = ["validate", "plan"]
}

inputs = {
  vpc_id             = dependency.networking.outputs.vpc_id
  private_subnet_ids = dependency.networking.outputs.private_subnet_ids
  instance_type      = "t3.large"
}
```

🔵 `run-all` — Apply the Entire Stack in Dependency Order

```
# Apply all stacks in prod in dependency order
cd infrastructure/prod
terragrunt run-all apply

# Terragrunt:
# 1. Reads all terragrunt.hcl files recursively
# 2. Builds dependency graph from dependency {} blocks
# 3. Applies in topological order: networking → security → data → compute
# 4. Parallel where no dependency exists (networking + security in
parallel)
# 5. Stops if any stack fails

# Equivalent for plan, destroy, output:
terragrunt run-all plan
terragrunt run-all destroy  # applies in REVERSE dependency order
terragrunt run-all output

# Filter to specific stacks:
terragrunt run-all apply --terragrunt-include-dir "networking,security"
```

🔵 Before/After Hooks

```
# terragrunt.hcl — lifecycle hooks
terraform {
  source = "../../modules/eks"

  before_hook "validate_prerequisites" {
    commands = ["apply", "plan"]
    execute  = ["bash", "-c", "aws eks list-clusters --region eu-west-1 |
jq ."]
  }

  after_hook "run_smoke_tests" {
    commands     = ["apply"]
    execute      = ["bash", "-c", "pytest tests/smoke/test_eks.py"]
    run_on_error = false   # only run if apply succeeded
  }

  error_hook "notify_on_failure" {
    commands  = ["apply"]
    execute   = ["bash", "-c",
      "curl -X POST $SLACK_WEBHOOK -d '{\"text\":\"Terraform apply failed
in compute!\"}'"]
    on_errors = [".*"]   # regex matching any error message
  }
}
```

🔵 Terragrunt vs Vanilla Terraform — Decision Framework

```
Use Vanilla Terraform when:
  ✅ Small team (< 5 engineers)
  ✅ Simple infrastructure (< 5 root modules)
  ✅ Terraform Cloud handles the orchestration
  ✅ Team new to Terraform — reduce cognitive overhead
  ✅ You don't want a Terragrunt learning curve

Use Terragrunt when:
  ✅ Large number of root modules (10+) with same backend config
  ✅ Need multi-stack apply with dependency ordering outside TFC
  ✅ Multi-account, multi-environment at scale
  ✅ Need before/after hooks in CLI workflow
  ✅ DRY inputs across many stacks is a real pain point
  ✅ Team is comfortable with the added complexity

Anti-pattern:
  ❌ Terragrunt + Terraform Cloud = redundant orchestration
  ❌ Deep Terragrunt nesting with 5+ levels of inheritance = debugging
nightmare
```

🔵 Short Interview Answer

> "Terragrunt is a wrapper around Terraform that solves three main pain points at scale: DRY backend configuration (define S3 backend once in root `terragrunt.hcl`, auto-generated in every module), `run-all` command that applies multiple stacks in dependency order (Terragrunt reads `dependency {}` blocks to build the graph), and before/after hooks for pre/post apply operations. The key Terragrunt concept is `find_in_parent_folders()` which walks up the directory tree to inherit configuration — each stack gets the common backend and provider config without copying it. Choose Terragrunt when you have 10+ root modules with repetitive backend config. Choose vanilla Terraform when you're using Terraform Cloud (which handles orchestration) or when the added complexity isn't justified."

Topic 76: ⚠ Large-Scale Terraform — Monorepo vs Polyrepo, State Splitting

---

🔵 What It Is (Simple Terms)

At large scale, decisions about repository structure and state file organization significantly impact team velocity, security, and reliability. This covers the tradeoffs between keeping all infrastructure in one repo vs

splitting across many.

🔵 Monorepo — All Infrastructure in One Repository

```
Structure:
  infrastructure/                ← single Git repository
  ├── modules/                   ← shared modules
  │   ├── vpc/
  │   ├── rds/
  │   └── eks/
  ├── environments/
  │   ├── dev/
  │   │   ├── networking/
  │   │   ├── compute/
  │   │   └── applications/
  │   ├── staging/
  │   └── prod/
  └── .github/workflows/

Advantages:
  ✅ Single source of truth — all infrastructure visible in one place
  ✅ Cross-stack changes in one PR — networking + compute change together
  ✅ Consistent versioning — modules and root configs always in sync
  ✅ Single CI/CD pipeline configuration
  ✅ Easier to enforce standards via CODEOWNERS + branch protection
  ✅ Simpler for smaller teams

Disadvantages:
  ❌ CI/CD gets complex — which stacks need to run when?
  ❌ Merge conflicts between teams working on different stacks
  ❌ Blast radius — a bad CI change can break all infrastructure CI
  ❌ Access control: if repo is shared, everyone sees all code
  ❌ Scales to ~50-100 stacks before becoming unwieldy
  ❌ Git history noisier — changes from all teams mixed
```

🔵 Polyrepo — Separate Repositories Per Team/Domain

```
Structure:
  platform-infrastructure/       ← Platform team repo
    modules/vpc/
    modules/eks/
    environments/prod/networking/
    environments/prod/compute/

  data-infrastructure/           ← Data team repo
    modules/rds/
    modules/kafka/
    environments/prod/databases/
```

```
  app-team-infrastructure/      ← App team repo
    environments/prod/services/
    environments/dev/services/

  shared-modules/               ← Shared library repo
    modules/vpc/
    modules/security-groups/

Advantages:
  ✅ Clear ownership — repo boundary = team boundary
  ✅ Independent access control per repo
  ✅ Simpler CI/CD per repo
  ✅ Each team deploys independently
  ✅ Scales to large organizations with many teams

Disadvantages:
  ❌ Cross-team changes require multiple PRs and coordination
  ❌ Module versioning across repos gets complex
  ❌ Shared module updates require updating all consumers
  ❌ Harder to see the full picture of infrastructure
  ❌ Each team must maintain their own CI/CD workflow
```

🔵 The Hybrid Pattern (Most Common at Scale)

```
shared-modules/          ← one repo, versioned releases
  modules/vpc/
  modules/rds/
  modules/eks/

platform-infra/          ← platform team (networking, k8s, shared
services)
  consumes: shared-modules@v2.1.0
  owns: prod/networking/*, prod/compute/*

team-a-infra/            ← app team A
  consumes: shared-modules@v2.1.0
  owns: prod/team-a/services/*

team-b-infra/            ← app team B
  consumes: shared-modules@v2.1.0, platform-infra outputs via SSM
  owns: prod/team-b/services/*

Cross-stack references:
  team-a-infra reads VPC IDs from SSM (platform-infra writes there)
  team-a-infra reads EKS cluster endpoint from SSM
  No direct terraform_remote_state — loose coupling via SSM
```

🔵 State Splitting at Scale — The Rules

```
Target: 50-200 resources per state file
(below 50 = excessive coordination overhead, above 200 = slow and risky)

Split dimensions:
  1. By environment: prod/*, staging/*, dev/* (always)
  2. By lifecycle: slow-changing (VPC, IAM) vs fast-changing (ECS, Lambda)
  3. By team ownership: platform/*, data/*, apps/*
  4. By blast radius: databases isolated, networking isolated

State that should ALWAYS be separate:
  → Networking (VPC, subnets) — everything depends on it, very stable
  → IAM / Security — high sensitivity, change carefully
  → Data stores (RDS, Redis) — stateful, blast radius is high
  → Compute (EKS, ASG) — changes frequently, depends on networking
  → Applications — changes most frequently, depends on compute

Good cross-stack reference via SSM (loose coupling):
  networking stack:
    resource "aws_ssm_parameter" "vpc_id" {
      name  = "/prod/networking/vpc_id"
      value = aws_vpc.main.id
    }

  compute stack:
    data "aws_ssm_parameter" "vpc_id" {
      name = "/prod/networking/vpc_id"
    }
    → No state dependency, changes in networking stack don't affect
compute plan
```

🔵 Mono vs Poly Decision Matrix

```
Team size:
  < 10 engineers  → monorepo (simpler)
  10-50 engineers → monorepo or hybrid (depends on team structure)
  50+ engineers   → polyrepo or hybrid (clear ownership boundaries needed)

Infrastructure complexity:
  < 5 environments, < 20 stacks → monorepo
  5+ environments, 20+ stacks   → hybrid or polyrepo

Compliance requirements:
  SOC2/PCI/HIPAA production isolation → separate repos (access control)

Team autonomy requirements:
  Teams want independent deploy cadence → polyrepo
  Teams want shared visibility         → monorepo
```

🔵 Short Interview Answer

> "For large-scale Terraform, the monorepo-vs-polyrepo decision comes down to team size and ownership boundaries. Under 10 engineers, a monorepo with clear directory structure is simpler. At 50+ engineers, polyrepo gives teams independent deployment, access control, and CI/CD. The most practical pattern at scale is a hybrid: shared modules in a versioned library repo, separate infra repos per team or domain. For state splitting: target 50-200 resources per state file, always split by environment, and further split by lifecycle (networking changes rarely, services change daily) and blast radius (databases isolated from compute). Cross-stack references via SSM Parameter Store rather than `terraform_remote_state` gives loose coupling — the compute stack doesn't need to plan when networking changes."

Topic 77: ➕ ⚠️ `terraform graph` — DAG Internals, Cycle Errors Deep Dive

🔵 What It Is (Simple Terms)

Terraform represents all resources and their dependencies as a **Directed Acyclic Graph (DAG)**. The `terraform graph` command outputs this graph in DOT format for visualization. Understanding the DAG helps debug dependency issues, cycle errors, and unexpected plan behavior.

🔵 The DAG — How Terraform Builds It

```
Terraform builds the DAG in three phases:

Phase 1: LOAD — parse all .tf files
  → Creates a node for every resource, data source, variable, output
  → Does NOT know about values yet — just structure

Phase 2: CONNECT — build edges (dependencies)
  → Implicit edges: any resource attribute reference creates a dependency
     aws_instance.web.subnet_id = aws_subnet.private.id
     → edge: aws_instance.web → aws_subnet.private (web depends on subnet)
  → Explicit edges: depends_on = [aws_iam_role.lambda]
     → edge: resource → aws_iam_role.lambda

Phase 3: WALK — execute in dependency order
  → Topological sort of the DAG
  → Resources with no dependencies: execute first (in parallel)
  → Resources with dependencies: execute after all their dependencies
complete
  → Default parallelism: 10 concurrent operations
```

🔵 Using `terraform graph`

```
# Generate the full dependency graph
terraform graph | dot -Tsvg > graph.svg
terraform graph | dot -Tpng > graph.png

# Generate plan graph (only changes)
terraform graph -type=plan | dot -Tsvg > plan-graph.svg

# Generate apply graph
terraform graph -type=apply | dot -Tsvg > apply-graph.svg

# Generate destroy graph (reversed)
terraform graph -type=plan-destroy | dot -Tsvg > destroy-graph.svg

# Install graphviz for dot rendering:
brew install graphviz    # macOS
apt install graphviz     # Ubuntu

# View in browser (interactive)
terraform graph | dot -Tsvg | open -a "Google Chrome" /dev/stdin
```

```
Graph output (DOT format):
  digraph {
    compound = "true"
    newrank  = "true"
    subgraph "root" {
      "[root] aws_instance.web (expand)"        -> "[root]
aws_subnet.private (expand)"
      "[root] aws_instance.web (expand)"        -> "[root]
aws_security_group.web (expand)"
      "[root] aws_subnet.private (expand)"      -> "[root] aws_vpc.main
(expand)"
      "[root] aws_security_group.web (expand)"  -> "[root] aws_vpc.main
(expand)"
      "[root] aws_vpc.main (expand)"            -> "[root]
provider[\"registry.terraform.io/hashicorp/aws\"]"
    }
  }
```

🔵 Cycle Error — What It Is

```
A cycle is a circular dependency in the DAG:
  Resource A depends on Resource B
```

```
  Resource B depends on Resource A
  → Neither can be created first → error

Error:
  Error: Cycle: aws_security_group.web, aws_security_group.database

Real scenario — circular SG references:
  resource "aws_security_group" "web" {
    egress {
      from_port       = 5432
      to_port         = 5432
      protocol        = "tcp"
      security_groups = [aws_security_group.database.id]  ← depends on
database SG
    }
  }

  resource "aws_security_group" "database" {
    ingress {
      from_port       = 5432
      to_port         = 5432
      protocol        = "tcp"
      security_groups = [aws_security_group.web.id]       ← depends on web
SG
    }
  }

  DAG: aws_security_group.web → aws_security_group.database →
aws_security_group.web
  ← CYCLE! Cannot resolve
```

🔵 Detecting and Breaking Cycles

```
# Terraform error output for cycles:
Error: Cycle: module.app.aws_security_group.web,
module.app.aws_security_group.db

# Step 1: Generate the graph to visualize
terraform graph 2>&1 | grep -A 50 "Cycle"

# Step 2: Look for the circular references in your config
# Usually: two resources referencing each other's IDs
```

```
# FIX: Break the cycle using aws_security_group_rule (separate resource)

# Create empty security groups first (no circular reference)
resource "aws_security_group" "web" {
  name   = "web-sg"
```

```
  vpc_id = aws_vpc.main.id
  # NO inline ingress/egress rules that reference other SGs
}

resource "aws_security_group" "database" {
  name   = "database-sg"
  vpc_id = aws_vpc.main.id
  # NO inline ingress/egress rules that reference other SGs
}

# Add rules separately — rules can reference both SGs (they already exist)
resource "aws_security_group_rule" "web_to_db" {
  type                     = "egress"
  from_port                = 5432
  to_port                  = 5432
  protocol                 = "tcp"
  security_group_id        = aws_security_group.web.id      # SG to add
rule to
  source_security_group_id = aws_security_group.database.id # reference
target SG
}

resource "aws_security_group_rule" "db_from_web" {
  type                     = "ingress"
  from_port                = 5432
  to_port                  = 5432
  protocol                 = "tcp"
  security_group_id        = aws_security_group.database.id
  source_security_group_id = aws_security_group.web.id
}

# DAG after fix:
# aws_security_group.web → aws_vpc.main
# aws_security_group.database → aws_vpc.main
# aws_security_group_rule.web_to_db → aws_security_group.web +
aws_security_group.database
# aws_security_group_rule.db_from_web → aws_security_group.web +
aws_security_group.database
# NO CYCLE ✅
```

🔵 Short Interview Answer

> "Terraform builds a Directed Acyclic Graph from all resource and data source declarations, with edges representing dependencies — both implicit (attribute references) and explicit (`depends_on`). The DAG enables Terraform to parallelize independent resources while respecting dependency ordering. `terraform graph | dot -Tsvg` renders it as an image. Cycle errors occur when two resources reference each other — most commonly with AWS security groups using inline ingress/egress rules that reference each other's IDs. The fix is always to separate resource creation from rule creation: create the security groups first (empty), then use `aws_security_group_rule`

> resources to add the cross-references. `aws_security_group_rule` resources depend on both SGs but the SGs don't depend on each other — cycle broken."

Topic 78: Testing — `terraform validate`, `fmt`, Native `terraform test` (1.6+)

🔵 What It Is (Simple Terms)

Terraform has a testing pyramid: syntax checking (`fmt`, `validate`), unit testing (`terraform test` with mocks), and integration testing (Terratest). Understanding all three and when each applies is essential for module quality.

🔵 `terraform fmt` — Code Formatting

```
# Check formatting (exit 1 if any files need reformatting)
terraform fmt -check
terraform fmt -check -recursive   # check all subdirectories

# Apply formatting
terraform fmt
terraform fmt -recursive

# Diff mode — see what would change without applying
terraform fmt -diff

# What fmt does:
# ✅ Indentation: 2 spaces
# ✅ Argument alignment: aligns = signs in blocks
# ✅ Block spacing: consistent newlines between blocks
# ✅ Quote normalization: "double quotes" consistently

# CI/CD usage:
- name: Check formatting
  run: terraform fmt -check -recursive
  # Fails PR if any .tf file is not formatted
```

🔵 `terraform validate` — Structural Validation

```
# Validate configuration syntax and internal consistency
terraform validate

# What it checks:
# ✅ Valid HCL syntax
```

```
# ✅ All required arguments present
# ✅ Argument types match expected types
# ✅ References exist (referenced resources/variables are declared)
# ✅ No undefined variables
# ✅ No circular references

# What it does NOT check:
# ❌ Provider API availability (no API calls)
# ❌ Whether resource values are valid (e.g., wrong AMI ID)
# ❌ State (runs without init if providers already downloaded)
# ❌ Variable values (uses null for unset variables)

# Requires init:
terraform init -backend=false   # init without configuring backend
terraform validate

# Returns:
# Success: Configuration is valid
# Error: Error: Unsupported argument / Missing required argument / etc.
```

🔵 `terraform test` — Native Testing Framework (Terraform 1.6+)

```
# tests/basic.tftest.hcl — test file format

# Variables for the test
variables {
  environment = "test"
  vpc_cidr    = "10.99.0.0/16"
  az_count    = 2
}

# Run 1: Create the infrastructure
run "vpc_created" {
  command = apply   # actually creates resources

  # Assertions on the apply output
  assert {
    condition     = aws_vpc.main.cidr_block == "10.99.0.0/16"
    error_message = "VPC CIDR block should be 10.99.0.0/16"
  }

  assert {
    condition     = aws_vpc.main.enable_dns_hostnames == true
    error_message = "DNS hostnames should be enabled"
  }

  assert {
    condition     = length(aws_subnet.private) == 2
    error_message = "Should create 2 private subnets"
  }
```

```
}

# Run 2: Verify outputs
run "check_outputs" {
  command = plan   # plan only, no new resources

  assert {
    condition     = output.vpc_id == aws_vpc.main.id
    error_message = "vpc_id output should match the VPC resource ID"
  }

  assert {
    condition     = length(output.private_subnet_ids) == 2
    error_message = "Should output 2 private subnet IDs"
  }
}
```

```
# Run tests
terraform test

# Output:
# tests/basic.tftest.hcl... in progress
#   run "vpc_created"... pass
#   run "check_outputs"... pass
# tests/basic.tftest.hcl... tearing down
# Success! 1 passed, 0 failed.

# Run specific test file
terraform test -filter=tests/basic.tftest.hcl

# Verbose output
terraform test -verbose
```

🔵 `terraform test` with Mock Providers (1.7+)

```
# tests/with_mocks.tftest.hcl

# Mock the AWS provider — no real AWS calls
mock_provider "aws" {
  mock_resource "aws_vpc" {
    defaults = {
      id         = "vpc-mock-12345678"
      cidr_block = "10.99.0.0/16"
    }
  }

  mock_resource "aws_subnet" {
    defaults = {
```

```
      id                = "subnet-mock-12345678"
      availability_zone = "eu-west-1a"
    }
  }
}

# With mock providers: runs in milliseconds, no real AWS account needed
run "test_module_logic" {
  command = plan   # plan with mocked provider

  assert {
    condition     = length(aws_subnet.private) == var.az_count
    error_message = "Number of subnets should equal az_count"
  }

  assert {
    condition     = aws_vpc.main.enable_dns_hostnames == true
    error_message = "DNS hostnames must be enabled"
  }
}
```

🔵 The Testing Pyramid

```
          ┌──────────┐
          │Terratest │  ← Slow (minutes), real infra, high confidence
          │Integration│    Real AWS resources created and destroyed
          └──────────┘

         ┌──────────┐
         │terraform  │  ← Medium (seconds-minutes), real API calls
         │test (apply)│    Real resources, cleaned up automatically
         └──────────┘

       ┌──────────┐
       │terraform test│  ← Fast (milliseconds), no real infra
       │(mock/plan)  │    Tests logic, expressions, outputs
       └──────────┘

      ┌──────────┐
      │validate + fmt │  ← Instant, syntax only, no API calls
      └──────────┘
```

🔵 Short Interview Answer

> "`terraform fmt -check` enforces consistent code style — fails CI if files aren't formatted. `terraform validate` checks syntax and reference validity without making API calls — catches undefined variables, wrong argument types, circular references. `terraform test` (1.6+) is the native testing framework — test files declare variables, run blocks with `plan` or `apply` commands, and `assert` blocks validate resource attributes and outputs. With `mock_provider` (1.7+), provider calls are intercepted and return mock values — enabling fast unit tests with no real infrastructure. For

> full integration testing where real resources are needed, Terratest provides a Go framework. I use all four in layers: fmt + validate in seconds, mocked tests for logic, apply tests for module integration, Terratest for end-to-end."

Topic 79: ⚠ Terratest — Go-Based Integration Testing for Modules

---

🔵 What It Is (Simple Terms)

Terratest is a Go library by Gruntwork for writing automated integration tests for Terraform modules — creating real infrastructure, running assertions against it, and destroying it. It provides the highest confidence that a module works correctly in a real environment.

🔵 Why Terratest Over `terraform test`

```
terraform test:
  ✅ Native, no extra tooling
  ✅ Mock providers = fast
  ⚠ Limited assertion capabilities
  ❌ Can't test actual AWS behavior (endpoint reachability, etc.)

Terratest:
  ✅ Full Go test capabilities (complex assertions, retries, HTTP checks)
  ✅ Tests real AWS behavior — can actually connect to the created RDS
  ✅ Can test multiple modules together (integration)
  ✅ Mature ecosystem — widely used in production
  ❌ Requires Go knowledge
  ❌ Real infrastructure = real cost during tests
  ❌ Real infrastructure = minutes to run
  ❌ Flaky AWS APIs can cause flaky tests
```

🔵 Basic Terratest Structure

```go
// test/vpc_test.go
package test

import (
    "testing"
    "fmt"

    "github.com/gruntwork-io/terratest/modules/aws"
    "github.com/gruntwork-io/terratest/modules/terraform"
```

```go
    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
)

func TestVpcModule(t *testing.T) {
    t.Parallel()   // run test cases in parallel for speed

    awsRegion := "eu-west-1"

    terraformOptions := &terraform.Options{
        // Path to the module to test
        TerraformDir: "../modules/vpc",

        // Input variables
        Vars: map[string]interface{}{
            "vpc_cidr":    "10.99.0.0/16",
            "environment": "test",
            "az_count":    2,
        },

        // Environment variables
        EnvVars: map[string]string{
            "AWS_DEFAULT_REGION": awsRegion,
        },
    }

    // Destroy at end of test (even if test fails)
    defer terraform.Destroy(t, terraformOptions)

    // Apply the module
    terraform.InitAndApply(t, terraformOptions)

    // Read outputs
    vpcId := terraform.Output(t, terraformOptions, "vpc_id")
    subnetIds := terraform.OutputList(t, terraformOptions,
"private_subnet_ids")

    // Assertions: verify the VPC exists in AWS
    vpc := aws.GetVpcById(t, vpcId, awsRegion)
    assert.Equal(t, "10.99.0.0/16", vpc.CidrBlock)

    // Verify DNS hostnames enabled
    require.True(t,
        aws.IsPublicDnsHostnamesEnabled(t, vpcId, awsRegion),
        "DNS hostnames should be enabled")

    // Verify correct number of subnets
    assert.Equal(t, 2, len(subnetIds))

    // Verify subnets are in different AZs (HA validation)
    azs := make(map[string]bool)
    for _, subnetId := range subnetIds {
        subnet := aws.GetSubnetById(t, subnetId, awsRegion)
        azs[subnet.AvailabilityZone] = true
```

```
    }
    assert.Equal(t, 2, len(azs), "Subnets should be in different AZs")
}
```

🔵 HTTP and Connectivity Testing

```
// test/web_service_test.go

func TestWebServiceEndpoint(t *testing.T) {
    t.Parallel()

    terraformOptions := &terraform.Options{
        TerraformDir: "../modules/web-service",
        Vars: map[string]interface{}{
            "environment": "test",
        },
    }
    defer terraform.Destroy(t, terraformOptions)
    terraform.InitAndApply(t, terraformOptions)

    // Get the load balancer URL from output
    lbUrl := terraform.Output(t, terraformOptions, "lb_url")

    // Test HTTP endpoint with retries (infrastructure takes time to be
ready)
    expectedStatus := 200
    http_helper.HttpGetWithRetry(
        t,
        fmt.Sprintf("https://%s/health", lbUrl),
        nil,                        // TLS config
        expectedStatus,
        "OK",                       // expected body content
        60,                         // max retries
        5 * time.Second,            // sleep between retries
    )
}
```

🔵 Table-Driven Tests

```
func TestVpcVariants(t *testing.T) {
    t.Parallel()

    testCases := []struct {
        name              string
        azCount           int
        enableNatGateway  bool
        expectedSubnets   int
```

```
    }{
        {"basic-2az", 2, false, 2},
        {"ha-3az-with-nat", 3, true, 3},
        {"single-az", 1, false, 1},
    }

    for _, tc := range testCases {
        tc := tc   // capture range variable for parallel tests
        t.Run(tc.name, func(t *testing.T) {
            t.Parallel()

            opts := &terraform.Options{
                TerraformDir: "../modules/vpc",
                Vars: map[string]interface{}{
                    "az_count":           tc.azCount,
                    "enable_nat_gateway": tc.enableNatGateway,
                    "vpc_cidr":           "10.99.0.0/16",
                    "environment":        "test",
                },
            }
            defer terraform.Destroy(t, opts)
            terraform.InitAndApply(t, opts)

            subnetIds := terraform.OutputList(t, opts,
"private_subnet_ids")
            assert.Equal(t, tc.expectedSubnets, len(subnetIds))

            if tc.enableNatGateway {
                natGwIds := terraform.OutputList(t, opts,
"nat_gateway_ids")
                assert.Equal(t, tc.azCount, len(natGwIds))
            }
        })
    }
}
```

🔵 Test Setup and Running

```
# Install dependencies
cd test
go mod init github.com/myorg/terraform-modules/test
go get github.com/gruntwork-io/terratest/modules/terraform
go get github.com/gruntwork-io/terratest/modules/aws
go get github.com/stretchr/testify/assert

# Run tests (requires AWS credentials)
go test -v -timeout 30m ./...

# Run specific test
go test -v -run TestVpcModule -timeout 30m ./...
```

```
# Run with race detection
go test -v -race -timeout 30m ./...
```

🔵 Short Interview Answer

> "Terratest is a Go testing library for infrastructure integration tests — it runs `terraform apply` against real cloud APIs, makes assertions using AWS SDK calls, then runs `terraform destroy`. The pattern: create `terraform.Options` with module path and input variables, `defer terraform.Destroy` to ensure cleanup even on failure, call `terraform.InitAndApply`, then read outputs and make assertions with testify. For services, `http_helper.HttpGetWithRetry` polls endpoints with backoff — critical because infrastructure takes time to become healthy. Tests run in parallel with `t.Parallel()` to keep overall time manageable. Terratest gives the highest confidence a module works correctly because it validates actual AWS behavior — not just that Terraform thinks the resource exists."

Topic 80: ➕ ⚠️ Mocking Providers in Tests — Patterns and Frameworks

---

🔵 What It Is (Simple Terms)

Mock providers intercept Terraform provider calls during testing and return predefined responses instead of calling real cloud APIs. This enables fast, repeatable tests with no real infrastructure and no cloud credentials.

🔵 Native Mock Providers in `terraform test` (1.7+)

```
# tests/unit/vpc_logic.tftest.hcl

# Mock the entire AWS provider
mock_provider "aws" {
  # Global defaults for all mocked resources
  mock_resource "aws_vpc" {
    defaults = {
      id                   = "vpc-mock12345"
      arn                  = "arn:aws:ec2:eu-west-1:123456789012:vpc/vpc-
mock12345"
      cidr_block           = "10.99.0.0/16"
      enable_dns_hostnames = true
      enable_dns_support   = true
      owner_id             = "123456789012"
    }
  }
```

```
  mock_resource "aws_subnet" {
    defaults = {
      id                   = "subnet-mock12345"
      arn                  = "arn:aws:ec2:eu-west-
1:123456789012:subnet/subnet-mock12345"
      availability_zone    = "eu-west-1a"
      cidr_block           = "10.99.0.0/24"
    }
  }

  mock_data "aws_availability_zones" {
    defaults = {
      names = ["eu-west-1a", "eu-west-1b", "eu-west-1c"]
      state = "available"
    }
  }
}

variables {
  vpc_cidr    = "10.99.0.0/16"
  environment = "test"
  az_count    = 3
}

# Tests run in milliseconds - no real AWS calls
run "verify_subnet_count" {
  command = plan

  assert {
    condition     = length(aws_subnet.private) == 3
    error_message = "Should create 3 private subnets when az_count=3"
  }
}

run "verify_naming" {
  command = plan

  assert {
    condition     = aws_vpc.main.tags["Name"] == "test-vpc"
    error_message = "VPC should be tagged with environment name"
  }
}
```

🔵 Mock Data Sources

```
# Mocking complex data sources
mock_provider "aws" {
  mock_data "aws_ami" {
    defaults = {
```

```
      id           = "ami-mock12345678"
      name         = "ubuntu-22.04-test"
      owner_id     = "099720109477"
      architecture = "x86_64"
    }
  }

  mock_data "aws_caller_identity" {
    defaults = {
      account_id = "123456789012"
      user_id    = "AIDAIOSFODNN7EXAMPLE"
      arn        = "arn:aws:iam::123456789012:user/test"
    }
  }

  mock_data "aws_region" {
    defaults = {
      name        = "eu-west-1"
      description = "EU (Ireland)"
    }
  }
}
```

🔵 Override vs Mock — The Difference

```
# MOCK: replace the entire provider — applies to all resources of that
type
mock_provider "aws" {
  mock_resource "aws_instance" {
    defaults = { id = "i-mock1234567890abcdef0" }
  }
}

# OVERRIDE: replace a specific resource instance — more surgical
override_resource {
  target = aws_instance.web
  values = {
    id            = "i-override-12345"
    instance_type = "t3.medium"
    private_ip    = "10.0.1.45"
  }
}

# Override a data source
override_data {
  target = data.aws_ami.ubuntu
  values = {
    id   = "ami-override-12345"
    name = "ubuntu-22.04-override"
  }
```

```
}

# Override a module's outputs
override_module {
  target  = module.vpc
  outputs = {
    vpc_id             = "vpc-override-12345"
    private_subnet_ids = ["subnet-override-1", "subnet-override-2"]
  }
}
```

🔵 Short Interview Answer

> "Terraform 1.7+ supports native mock providers in test files. `mock_provider` blocks intercept provider calls and return predefined values — tests run in milliseconds with no cloud credentials. `mock_resource` provides defaults for all resources of a type; `mock_data` mocks data sources. For more surgical control, `override_resource` replaces a specific resource instance, and `override_module` replaces an entire module's outputs. This enables unit testing of module logic — expression transformations, conditional resources, output calculations — without the latency and cost of real infrastructure. Mock tests sit at the bottom of the testing pyramid: fast enough to run on every commit, complemented by real-infrastructure tests (terraform test with apply, Terratest) before releases."

Topic 81: Contract Testing Between Modules — Interface/API Design

---

🔵 What It Is (Simple Terms)

Contract testing verifies that a module's **interface** (its input variables and output values) meets the expectations of modules that consume it — detecting breaking changes before they cause downstream failures.

🔵 Designing Module Contracts

```
# A module's contract = its variables.tf + outputs.tf

# modules/vpc/variables.tf — the contract callers must honor
variable "vpc_cidr" {
  type        = string
  description = "CIDR block for the VPC. Must be /16 to /24."
  validation {
    condition     = can(cidrhost(var.vpc_cidr, 0))
    error_message = "vpc_cidr must be a valid CIDR block."
```

```
  }
  validation {
    condition     = tonumber(split("/", var.vpc_cidr)[1]) <= 24
    error_message = "vpc_cidr prefix must be /16 to /24."
  }
}

# modules/vpc/outputs.tf — the contract the module honors to callers
output "vpc_id" {
  description = "The ID of the created VPC. Format: vpc-xxxxxxxxxxxxxxxxx"
  value       = aws_vpc.main.id
}

output "private_subnet_ids" {
  description = "List of private subnet IDs. Length equals az_count."
  value       = aws_subnet.private[*].id
}
```

🔵 Breaking vs Non-Breaking Changes

```
NON-BREAKING changes (safe to release as minor version):
  ✅ Add new optional variable (has default)
  ✅ Add new output
  ✅ Change resource defaults (no interface change)
  ✅ Bug fix that doesn't change output values

BREAKING changes (require major version bump + moved blocks):
  ❌ Remove a variable
  ❌ Rename a variable
  ❌ Change a variable's type (string → list)
  ❌ Remove an output
  ❌ Rename an output
  ❌ Change an output's type (string → object)
  ❌ Change resource structure (count → for_each changes state addresses)
  ❌ Add required variable (no default) to existing module
```

🔵 Contract Test Implementation

```
# tests/contract/outputs_contract.tftest.hcl
# Verify the module's outputs match the expected contract

mock_provider "aws" {
  mock_resource "aws_vpc"    { defaults = { id = "vpc-mock12345" } }
  mock_resource "aws_subnet" { defaults = { id = "subnet-mock" } }
  mock_data "aws_availability_zones" {
    defaults = { names = ["eu-west-1a", "eu-west-1b"] }
  }
```

```
}

variables {
  vpc_cidr    = "10.0.0.0/16"
  environment = "test"
  az_count    = 2
}

run "output_contract" {
  command = plan

  # Contract: vpc_id must be a string matching VPC ID format
  assert {
    condition     = can(regex("^vpc-[0-9a-f]+$", output.vpc_id))
    error_message = "vpc_id output must be a VPC ID (vpc-xxxxxxxxx
format)"
  }

  # Contract: private_subnet_ids must be a list
  assert {
    condition     = length(output.private_subnet_ids) > 0
    error_message = "private_subnet_ids must be a non-empty list"
  }

  # Contract: subnet count must equal az_count
  assert {
    condition     = length(output.private_subnet_ids) == var.az_count
    error_message = "Number of private subnets must equal az_count"
  }

  # Contract: all subnet IDs must match subnet ID format
  assert {
    condition = alltrue([
      for id in output.private_subnet_ids :
      can(regex("^subnet-[0-9a-f]+$", id))
    ])
    error_message = "All subnet IDs must be valid subnet ID format"
  }
}
```

🔵 Short Interview Answer

> "Contract testing for modules verifies that a module's interface — its input variables and output values — meets the expectations of its consumers. A module's contract is defined by `variables.tf` (what callers must provide) and `outputs.tf` (what the module guarantees to return). Breaking changes include removing or renaming variables/outputs, changing their types, or adding required variables to an existing interface. Contract tests use mock providers to quickly verify output shapes and types without real infrastructure — they check that `vpc_id` is a string matching VPC format, `private_subnet_ids` is a list with length equal to `az_count`, etc. When releasing a module version, contract tests give callers confidence that upgrading won't break their code."

Topic 82: Testing Module Contracts & Output Validation

🔵 Output Validation Patterns

```
# Deep output validation using terraform test assertions

run "validate_all_outputs" {
  command = plan

  # Type validation
  assert {
    condition     = can(tostring(output.vpc_id))
    error_message = "vpc_id must be convertible to string"
  }

  # Structural validation
  assert {
    condition = alltrue([
      for subnet_id in output.private_subnet_ids :
      can(regex("^subnet-[0-9a-f]{8,17}$", subnet_id))
    ])
    error_message = "All private subnet IDs must be valid format"
  }

  # Cross-output consistency
  assert {
    condition     = output.vpc_cidr == var.vpc_cidr
    error_message = "Output vpc_cidr must match input var.vpc_cidr"
  }

  # Completeness: all expected keys present in map output
  assert {
    condition = alltrue([
      for key in ["vpc_id", "private_subnet_ids", "public_subnet_ids"] :
      contains(keys(output.network_config), key)
    ])
    error_message = "network_config output must contain all required keys"
  }
}
```

🔵 Integration Test vs Contract Test — The Distinction

```
Contract Test (fast, mocked):
  → Does the module PRODUCE the right SHAPE of outputs?
  → Are output types correct? Are required keys present?
  → Does the module honor its variable validation rules?
  → Runs in milliseconds with mock providers
  → Run on every commit

Integration Test (slow, real):
  → Does the module WORK CORRECTLY in a real environment?
  → Does the created VPC actually work? Can instances route through it?
  → Are security groups effective? Does RDS accept connections?
  → Runs in minutes, costs real money
  → Run on PRs to main / pre-release
```

🔵 Short Interview Answer

> "Contract tests validate output shapes and types using mock providers — fast enough for every commit. Integration tests validate actual behavior with real infrastructure. For output validation: use `can()` with type conversion functions to check types, `regex()` for format validation, and cross-output assertions for consistency (output CIDR matches input CIDR). The key distinction is coverage depth vs speed: contract tests catch interface regressions in milliseconds, integration tests catch behavioral regressions in minutes. Both are needed — neither replaces the other."

Topic 83: `TF_LOG` Levels — Debugging Provider and Core Issues

🔵 What It Is (Simple Terms)

`TF_LOG` controls Terraform's debug output verbosity. When a plan or apply behaves unexpectedly — a resource keeps getting recreated, an API call fails, a provider crashes — debug logging reveals the internal HTTP calls, provider plugin communication, and DAG evaluation.

🔵 Log Levels and What They Show

```
# Available levels (most to least verbose):
export TF_LOG=TRACE   # everything — gRPC messages, plugin protocol frames
export TF_LOG=DEBUG   # HTTP requests/responses, plugin calls, DAG
evaluation
export TF_LOG=INFO    # informational messages from providers
export TF_LOG=WARN    # deprecation warnings, non-fatal issues
export TF_LOG=ERROR   # error conditions only
export TF_LOG=OFF     # disable logging (default)
```

```
# Save logs to file (stdout is too noisy for most debugging)
export TF_LOG=DEBUG
export TF_LOG_PATH=./terraform-debug.log
terraform plan 2>&1   # stderr is the log channel

# Separate core and provider logs (Terraform 0.15+)
export TF_LOG_CORE=DEBUG       # Terraform core (DAG, state, planning)
export TF_LOG_PROVIDER=DEBUG   # Provider plugin only (AWS API calls)
export TF_LOG_PROVIDER=INFO    # Provider info, core stays at default
```

🔵 What Each Level Shows

```
# TF_LOG=DEBUG — most useful for debugging

## Provider API calls:
2024/01/15 10:30:01 [DEBUG] provider.terraform-provider-aws: Request:
  POST https://ec2.eu-west-1.amazonaws.com/
  Action=DescribeInstances&InstanceId.1=i-0abc123

## Response:
2024/01/15 10:30:01 [DEBUG] provider.terraform-provider-aws: Response:
  Status: 200 OK
  Body: <DescribeInstancesResponse>...</DescribeInstancesResponse>

## Terraform Core — DAG evaluation:
2024/01/15 10:30:00 [DEBUG] vertex "module.app.aws_instance.web": starting
walk
2024/01/15 10:30:00 [DEBUG] vertex "module.app.aws_subnet.private":
starting walk
2024/01/15 10:30:01 [DEBUG] vertex "module.app.aws_vpc.main": complete

## State operations:
2024/01/15 10:30:00 [DEBUG] states: reading state from s3://terraform-
state/prod/...
2024/01/15 10:30:00 [DEBUG] states: state is serial 47
```

🔵 Debugging Common Issues with TF_LOG

```
# Issue 1: Resource keeps showing changes every plan (perpetual diff)
export TF_LOG=DEBUG
terraform plan 2>&1 | grep -A 5 "aws_instance.web"
# Look for: what attribute the provider returns vs what state has
# Common cause: provider normalizes the value differently than config

# Issue 2: Plan takes too long
export TF_LOG=INFO
```

```
terraform plan 2>&1 | grep "Refreshing state"
# Shows which resources are being refreshed and how long each takes
# Identify slow data sources or throttled API calls

# Issue 3: Authentication failures
export TF_LOG=DEBUG
terraform init 2>&1 | grep -i "auth\|credential\|token\|role"
# Shows which credential chain is being used
# Shows STS AssumeRole calls and responses

# Issue 4: Provider plugin crashes
export TF_LOG=TRACE
terraform apply 2>&1 | head -200 > crash-debug.log
# Look for: panic messages, gRPC communication errors
# See Topic 86 for crash analysis

# Issue 5: State locking issues
export TF_LOG=DEBUG
terraform apply 2>&1 | grep -i "lock\|dynamo"
# Shows DynamoDB PutItem/GetItem calls for lock acquisition
```

🔵 Grep Patterns for Common Issues

```
# Show all HTTP requests
TF_LOG=DEBUG terraform plan 2>&1 | grep "^2.*Request:"

# Show all HTTP responses and status codes
TF_LOG=DEBUG terraform plan 2>&1 | grep "Status:"

# Show rate limiting / throttling
TF_LOG=DEBUG terraform plan 2>&1 | grep -i
"throttl\|TooManyRequests\|RequestLimitExceeded"

# Show what changed vs state
TF_LOG=DEBUG terraform plan 2>&1 | grep -i "prior state\|planned
state\|diff"

# Show module evaluation order
TF_LOG=DEBUG terraform plan 2>&1 | grep "vertex\|walk\|complete"

# Show provider plugin startup
TF_LOG=DEBUG terraform init 2>&1 | grep "plugin\|provider\|reattach"
```

🔵 Short Interview Answer

> "`TF_LOG` controls Terraform's debug output. `DEBUG` is the most useful level — shows all HTTP requests to cloud APIs, provider responses, DAG vertex evaluation, and state operations. `TRACE` adds gRPC plugin protocol messages. Set `TF_LOG_PATH` to save logs to a file instead of stdout

> pollution. In Terraform 0.15+, `TF_LOG_CORE` and `TF_LOG_PROVIDER` let you set different levels for Terraform core and provider plugins separately. Common debugging patterns: perpetual diff → check what value the provider returns in DEBUG logs; slow plans → grep for 'Refreshing state' to identify slow resources; auth failures → grep for credential/role/token; throttling → grep for TooManyRequests. Always save to file — DEBUG output is thousands of lines."

Topic 84: ⚠ State Corruption — Causes, Recovery, Prevention

🔵 What It Is (Simple Terms)

State corruption is when the Terraform state file becomes inconsistent, invalid JSON, or diverges from reality in a way that causes plans to fail or produce incorrect results. It's a serious incident — recovery must be careful and methodical.

🔵 Causes of State Corruption

```
Cause 1: Concurrent applies without locking
  Two applies read serial=47, both make changes, second write corrupts
  Prevention: Always configure DynamoDB locking with S3 backend

Cause 2: Interrupted apply
  Apply was killed mid-execution (SIGKILL, OOM, network failure)
  Resources partially created — state only reflects pre-apply state
  Some real resources exist but are not in state (orphans)

Cause 3: Manual state editing
  Someone edited the state JSON directly — syntax error or wrong values
  Prevention: Never edit state manually — use terraform state commands

Cause 4: Force-pushing old state
  terraform state push -force backup-from-yesterday.tfstate
  Overwrites newer serial — state now missing recent changes
  Prevention: Only force-push if you verified it's newer than current

Cause 5: Provider bugs
  Provider writes incorrect values to state after apply
  Attributes stored don't match real resource — causes perpetual diff
  Mitigation: Provider upgrades, terraform refresh

Cause 6: Backend misconfiguration
  Using wrong key/region — applying to wrong state accidentally
  Prevention: Validate backend config before every init
```

🔵 Detecting State Corruption

```
# Symptom 1: terraform plan fails with JSON errors
Error: Failed to read the updated state: ...
  unexpected character at index 0 (line 1, col 0): invalid character...
# → State file contains invalid JSON → full corruption

# Symptom 2: Plan shows unexpected destroys for all resources
Plan: 0 to add, 0 to change, 47 to destroy.
# → State file has wrong resources / empty resources section

# Symptom 3: Serial mismatch
Error: state snapshot was created by Terraform vX.Y.Z, which is
incompatible...
# → State pushed from wrong version

# Symptom 4: Resources exist in AWS but plan shows "will be created"
# → State entry is missing for existing resources
# → Partial corruption — some entries missing

# Diagnosis:
terraform state pull | jq '.' 2>&1
# If jq fails → invalid JSON → full corruption
# If jq succeeds → inspect .serial, .resources, .lineage
terraform state pull | jq '.resources | length'  # count resource entries
terraform state pull | jq '.serial'               # check serial number
```

🔵 Recovery Procedures

```
# ────────────────────────────────────────────────────────────────
# Case 1: State is invalid JSON (full corruption)
# ────────────────────────────────────────────────────────────────

# Step 1: Get the last valid version from S3 versioning
aws s3api list-object-versions \
  --bucket mycompany-terraform-state \
  --prefix prod/networking/terraform.tfstate \
  --query 'Versions[*].[VersionId,LastModified]' \
  --output table

# Step 2: Download the last known-good version
aws s3api get-object \
  --bucket mycompany-terraform-state \
  --key prod/networking/terraform.tfstate \
  --version-id "pre-corruption-version-id" \
  ./terraform.tfstate.backup

# Step 3: Verify the backup is valid JSON
cat terraform.tfstate.backup | jq '.serial, (.resources | length)'
```

```
# Step 4: Push the backup as the new state
terraform state push terraform.tfstate.backup
# Note: may need -force if serial is lower than corrupted state

# Step 5: Run plan to assess any drift from the restored state
terraform plan
# Expect: some resources may show changes if the backup predates recent
applies

# ────────────────────────────────────────────────────────────────────
# Case 2: Resources exist in AWS but missing from state (orphans)
# ────────────────────────────────────────────────────────────────────

# Step 1: List what's in AWS vs what's in state
terraform state list > state-resources.txt
# Manually compare with AWS Console or CLI

# Step 2: Import orphaned resources back into state
terraform import aws_instance.web i-0abc123def456789
terraform import aws_security_group.app sg-0abc123

# Step 3: Verify plan shows 0 changes for imported resources
terraform plan
# Small differences may appear — update config to match reality

# ────────────────────────────────────────────────────────────────────
# Case 3: State shows resources that don't exist in AWS
# ────────────────────────────────────────────────────────────────────

# Step 1: Identify the phantom resource
terraform plan
# Shows: aws_instance.web must be created (but it's already in state)
# OR: shows error fetching resource that doesn't exist

# Step 2: Remove the phantom from state
terraform state rm aws_instance.web

# Step 3: If resource should exist: import it or recreate it
# If resource shouldn't exist: leave it removed, delete from config
```

🔵 Prevention Checklist

```
✅ Always configure DynamoDB locking for S3 backend
✅ Enable S3 bucket versioning before first state write
✅ Never run multiple terraform apply simultaneously (CI/CD concurrency
controls)
✅ Never edit state files directly with a text editor
✅ Always backup state before manual state operations (state pull >
backup.tfstate)
```

```
✅ Use terraform state mv/rm — never JSON edits
✅ In CI/CD: only one apply job can run per workspace at a time
✅ After interrupted applies: always run plan before next apply
✅ Set state backup retention (S3 lifecycle: keep 90 days of versions)
✅ Test state restoration procedure quarterly (don't discover it's broken
during an incident)
```

🔵 Short Interview Answer

> "State corruption most commonly comes from concurrent applies without locking, interrupted applies, or manual state editing. Detection: `terraform state pull | jq '.'` — if jq fails, invalid JSON means full corruption. Recovery depends on severity: full JSON corruption → restore from S3 versioned backup with `state push`; missing state entries → `terraform import` orphaned resources back; phantom state entries → `terraform state rm`. Prevention requires: DynamoDB locking (prevents concurrent corruption), S3 versioning (enables recovery), CI/CD concurrency controls (only one apply per workspace), and absolute prohibition on direct JSON editing. The golden rule: any state manipulation must start with `state pull > backup.tfstate` and end with `plan` to verify 0 unexpected changes."

Topic 85: `terraform force-unlock` Advanced Scenarios

🔵 Advanced Lock Failure Patterns

```
# Scenario 1: Lock held by a CI/CD job that finished but lock wasn't
released
# (network partition during apply cleanup)

# Identify: Lock is old (hours/days), no job currently running
# Action: Safe to force-unlock

# Find the lock ID from the error message
terraform apply
# Error: Error acquiring the state lock
# Lock Info:
#   ID:      f3b45c2d-1234-5678-abcd-ef0123456789
#   Created: 2024-01-15 02:30:00 UTC    ← 8 hours ago

terraform force-unlock f3b45c2d-1234-5678-abcd-ef0123456789

# ──────────────────────────────────────────────────────────────
# Scenario 2: Lock held by Terraform version that crashed
# (Terraform panicked and couldn't release lock)
# ──────────────────────────────────────────────────────────────
```

```
# Same procedure: check age, verify no running job, force-unlock
# After unlock: run plan to check state consistency
# Crashed Terraform may have written partial state

# ────────────────────────────────────────────────────────────────────
# Scenario 3: Multiple stale locks across many workspaces
# (CI/CD system failure that killed many jobs simultaneously)
# ────────────────────────────────────────────────────────────────────

# List all locks in DynamoDB
aws dynamodb scan \
  --table-name terraform-state-locks \
  --query 'Items[*].LockID.S' \
  --output table

# Get details of each lock
aws dynamodb scan \
  --table-name terraform-state-locks \
  --output json | jq '.Items[] | {
    LockID: .LockID.S,
    Created: (.Info.S | fromjson | .Created),
    Who: (.Info.S | fromjson | .Who)
  }'

# Batch unlock after verifying all are stale:
# (no built-in batch command — must unlock one at a time)
for LOCK_ID in $(aws dynamodb scan \
  --table-name terraform-state-locks \
  --query 'Items[*].Info.S' \
  --output text | jq -r '.ID'); do
  echo "Unlocking: $LOCK_ID"
  # Must be in correct workspace directory for each
done

# Or: direct DynamoDB batch delete (emergency only)
aws dynamodb delete-item \
  --table-name terraform-state-locks \
  --key '{"LockID": {"S": "mycompany-tf-
state/prod/networking/terraform.tfstate"}}'
```

🔵 Force-Unlock After Partial Apply

```
# Scenario: Apply was running and got killed mid-apply
# State may be inconsistent — some resources created, state not updated

# Step 1: Force unlock (apply is definitely not running)
terraform force-unlock <lock-id>

# Step 2: Pull state and inspect serial
```

```
terraform state pull | jq '.serial'
# Compare to what you expect — if lower than expected, state may be behind

# Step 3: List resources in state
terraform state list
# Compare to what you know exists in AWS

# Step 4: Import any orphaned resources
# (resources that exist in AWS but not in state)
terraform import aws_instance.web i-0abc123

# Step 5: Run plan — expect some drift
terraform plan
# Carefully review — don't blindly apply

# Step 6: Reconcile differences
# Option A: update config to match reality
# Option B: apply to revert manual/partial changes to config
```

🔵 Prevention — Graceful Shutdown Handling

```
# GitHub Actions: graceful termination handling
jobs:
  apply:
    steps:
      - name: Terraform Apply
        run: |
          # Trap signals — run cleanup even if job is cancelled
          cleanup() {
            echo "Job interrupted — Terraform may hold a lock"
            echo "Lock will expire when the process terminates"
            # The lock is released automatically when the process exits
            # even if killed with SIGTERM
          }
          trap cleanup SIGTERM SIGINT

          terraform apply -auto-approve

    # Note: SIGKILL (kill -9) bypasses traps and leaves lock held
    # Only cancelling jobs via SIGTERM (graceful) triggers cleanup
```

🔵 Short Interview Answer

> "For production force-unlock scenarios: always start by verifying the lock is genuinely stale — check the `Created` timestamp in the error message and confirm no CI/CD jobs are currently running. For batch stale locks from a CI/CD outage, scan DynamoDB directly to list all lock records and verify each is stale. After any force-unlock following a partial apply, inspect state with `terraform state`

> `pull`, compare to what actually exists in AWS using `terraform state list`, import any orphaned resources with `terraform import`, then run `terraform plan` to assess inconsistency before the next apply. The direct DynamoDB delete approach (`aws dynamodb delete-item`) is the nuclear option for when Terraform itself can't connect to the backend."

Topic 86: ➕ Provider Plugin Crash Debugging — Reading Crash Logs, Reporting

---

🔵 What It Is (Simple Terms)

Terraform providers are separate processes communicating with Terraform core via gRPC. When a provider crashes (Go panic, nil pointer, assertion failure), it terminates the subprocess and leaves a crash log. Reading these logs helps distinguish provider bugs from config issues.

🔵 What a Provider Crash Looks Like

```
terraform apply

# Normal output:
# aws_instance.web: Creating...

# Suddenly:
│ Error: The terraform-provider-aws plugin crashed!
│
│ Unfortunately, this won't provide the usual helpful error message.
However,
│ this output may still be helpful in understanding which part of
Terraform was
│ affected. Note that the plugin output below was sent to stderr, not
stdout.
│
│ --- PLUGIN OUTPUT ---
│
│ panic: runtime error: invalid memory address or nil pointer dereference
│ [signal SIGSEGV: segmentation violation code=0x1 addr=0x0 pc=0x...]
│
│ goroutine 1 [running]:
│ github.com/hashicorp/terraform-provider-
aws/internal/service/ec2.resourceInstanceCreate(...)
│       /home/runner/work/terraform-provider-aws/.../instance.go:123
+0x456
│ github.com/hashicorp/terraform-provider-aws/internal/service/ec2.
(*resourceInstance).Create(...)
│       /home/runner/work/terraform-provider-aws/.../instance.go:78 +0x234
│
│ --- END PLUGIN OUTPUT ---
```

🔵 Reading Crash Logs

```
# Enable full crash capture
export TF_LOG=TRACE
export TF_LOG_PATH=./provider-crash.log
terraform apply 2>&1

# Key sections to read in crash log:

# 1. The panic message — what went wrong
grep "panic:" provider-crash.log
# panic: runtime error: invalid memory address or nil pointer dereference

# 2. The goroutine stack trace — WHERE in the provider code
grep -A 30 "goroutine 1 \[running\]" provider-crash.log
# Shows the exact file + line number in provider source code
# github.com/hashicorp/terraform-provider-
aws/internal/service/ec2/instance.go:123

# 3. What Terraform was doing when the crash occurred
grep -B 20 "panic:" provider-crash.log
# Shows which resource, which operation (Create/Read/Update/Delete)

# 4. Provider version
grep "provider.terraform-provider-aws" provider-crash.log | head -5
# Shows which version of the provider crashed

# 5. Request context — what data was sent to the provider
grep "Request:" provider-crash.log | head -20
```

🔵 Triage — Provider Bug vs Config Issue

```
Likely PROVIDER BUG:
  → Crash happens with valid config and valid attribute values
  → Crash is reproducible with the same config
  → Goroutine stack points to provider internals (nil pointer in provider
code)
  → Other users report same crash in GitHub Issues

Likely CONFIG ISSUE:
  → Crash happens with unusual/extreme input values
  → Using undocumented combinations of arguments
  → Using deprecated arguments
  → Error before crash mentions validation failure

Likely VERSION ISSUE:
```

```
  → Crash works on previous provider version
  → Crash introduced in a recent provider upgrade
  → Check GitHub provider releases for related issues
```

🔵 Reporting a Provider Bug

```
# 1. Isolate the minimal reproduction case
# Create the simplest possible config that reproduces the crash

# 2. Gather information:
terraform version
# Terraform v1.6.0
# + provider registry.terraform.io/hashicorp/aws v5.31.0

# 3. Search existing issues
# https://github.com/hashicorp/terraform-provider-aws/issues
# Search: "panic instance create" or the specific function from the stack
trace

# 4. File a bug report with:
# - Terraform version
# - Provider version
# - Minimal reproduction config (sanitize sensitive values)
# - Full crash log (sanitize account IDs, region, resource IDs)
# - Expected behavior
# - Actual behavior (the crash)
```

🔵 Workarounds While Waiting for a Fix

```
# Workaround 1: Pin to previous working version
# .terraform.lock.hcl — downgrade provider
terraform {
  required_providers {
    aws = {
      version = "= 5.29.0"   # last version before the crash
    }
  }
}
terraform init -upgrade

# Workaround 2: Use lifecycle to avoid the crashing operation
resource "aws_instance" "web" {
  lifecycle {
    ignore_changes = [tags]   # if crash is in tag update code path
  }
}
```

```
# Workaround 3: Split the operation
# Instead of create with all attributes at once
# Create minimal → update with additional config in a separate apply
```

🔵 Short Interview Answer

> "Provider crashes manifest as 'The plugin crashed!' errors with a goroutine stack trace. To debug: enable `TF_LOG=TRACE` with `TF_LOG_PATH` to capture the full log. Read the panic message (what failed), the goroutine stack (where in provider code), and the preceding log lines (what operation was being performed). Search the provider's GitHub Issues for the function name from the stack trace. Distinguish provider bugs (nil pointer in provider code, reproducible, others report it) from config issues (unusual input values, deprecated arguments). Workarounds: pin to the last working provider version, use `ignore_changes` to avoid the crashing code path, or split the operation into create + update steps. File the bug with minimal reproduction config, both version numbers, and the sanitized crash log."

Topic 87: ➕ ⚠ Cycle Errors — Deep Internal Understanding, How DAG Detects Them

---

🔵 What It Is (Simple Terms)

Cycle errors are the result of circular dependencies in Terraform's resource graph — Resource A depends on B, B depends on A. Terraform's graph walk algorithm (Tarjan's strongly connected components or Kahn's algorithm) detects these during the planning phase.

🔵 How Terraform's DAG Detects Cycles

```
Terraform uses a topological sort (DFS-based) to walk the graph.
The algorithm is based on Tarjan's SCC or a similar depth-first search:

1. Start from root node
2. Mark current node as "in progress"
3. Visit all dependency nodes (edges)
4. If we visit a node already marked "in progress" → CYCLE DETECTED
5. Otherwise: mark node as "complete"

Pseudocode:
  func walk(node):
    state[node] = IN_PROGRESS
    for each dependency of node:
      if state[dependency] == IN_PROGRESS:
        report_cycle(node, dependency)   ← CYCLE!
      elif state[dependency] == NOT_VISITED:
```

```
        walk(dependency)
    state[node] = COMPLETE

The error message includes the CYCLE path:
  Error: Cycle: A, B, C, A
  → A depends on B, B depends on C, C depends on A → cycle
```

🔵 Common Cycle Patterns and Fixes

```
# ── Pattern 1: Security Group circular reference ──────────────────────
# (Most common cycle error in AWS)

# CAUSES CYCLE:
resource "aws_security_group" "web" {
  egress {
    security_groups = [aws_security_group.db.id]   # web → db
  }
}
resource "aws_security_group" "db" {
  ingress {
    security_groups = [aws_security_group.web.id]  # db → web
  }
}
# Cycle: aws_security_group.web → aws_security_group.db →
aws_security_group.web

# FIX: Separate rules from group creation
resource "aws_security_group" "web" { name = "web"; vpc_id =
aws_vpc.main.id }
resource "aws_security_group" "db"  { name = "db";  vpc_id =
aws_vpc.main.id }

resource "aws_security_group_rule" "web_egress_to_db" {
  type                     = "egress"
  security_group_id        = aws_security_group.web.id
  source_security_group_id = aws_security_group.db.id
  from_port = 5432; to_port = 5432; protocol = "tcp"
}

resource "aws_security_group_rule" "db_ingress_from_web" {
  type                     = "ingress"
  security_group_id        = aws_security_group.db.id
  source_security_group_id = aws_security_group.web.id
  from_port = 5432; to_port = 5432; protocol = "tcp"
}
# NO CYCLE: both rules depend on both SGs, but SGs don't depend on each
other ✅

# ── Pattern 2: IAM role and policy cycle ──────────────────────────────
```

```
# CAUSES CYCLE (less common but possible with bad depends_on usage):
resource "aws_iam_role" "lambda" {
  depends_on = [aws_iam_policy.lambda]   # explicit bad dependency
}
resource "aws_iam_policy" "lambda" {
  depends_on = [aws_iam_role.lambda]     # creates cycle
}
# Fix: remove the circular depends_on (there's no real dependency here)

# ── Pattern 3: Module self-reference ──────────────────────────────

# CAUSES CYCLE:
module "app" {
  source = "./modules/app"
  vpc_id = module.app.vpc_id   # module referencing its own output!
}
# Fix: reference the correct source (module.networking.vpc_id, not
module.app)

# ── Pattern 4: Data source cycle ──────────────────────────────────

# CAUSES CYCLE (unusual):
data "aws_iam_policy_document" "lambda" {
  statement {
    resources = [aws_lambda_function.main.arn]   # data → lambda
  }
}
resource "aws_lambda_function" "main" {
  role = aws_iam_role.lambda.arn
}
resource "aws_iam_role" "lambda" {
  inline_policy {
    policy = data.aws_iam_policy_document.lambda.json  # lambda → data
  }
}
# data → lambda → iam_role → data (via inline policy) = CYCLE

# Fix: Use aws_iam_role_policy resource instead of inline_policy
resource "aws_iam_role" "lambda" {
  # no inline_policy referencing the data source
}
resource "aws_iam_role_policy" "lambda" {
  role   = aws_iam_role.lambda.id
  policy = data.aws_iam_policy_document.lambda.json
  # iam_role_policy depends on: data source + iam_role (no cycle) ✅
}
```

🔵 Debugging Cycle Errors

```
# Terraform's error message gives you the cycle path:
Error: Cycle: module.app.aws_security_group.web,
module.app.aws_security_group.database

# Step 1: Identify all resources in the cycle from the error
# aws_security_group.web and aws_security_group.database

# Step 2: Generate graph and look at just those nodes
terraform graph | grep -E "security_group\.(web|database)" > sg-cycle.dot

# Step 3: Find why they reference each other
grep -n "security_group\.(web|database)" modules/app/*.tf
# Look for: one resource block's attributes referencing the other
resource's ID

# Step 4: Apply the appropriate fix pattern
# (separate creation from rule configuration as shown above)

# Step 5: Verify fix — graph should no longer show cycle
terraform validate   # validates no cycles before any API calls
```

🔵 `depends_on` Cycles

```
# depends_on can create cycles if used carelessly

# A common bad pattern — explicit depends_on creating unnecessary cycles
resource "aws_lambda_function" "processor" {
  depends_on = [aws_sqs_queue.input]   # reasonable — IAM propagation
}

resource "aws_sqs_queue" "input" {
  depends_on = [aws_lambda_function.processor]   # ← BAD! circular
}

# Fix: Remove circular depends_on
# The SQS queue doesn't actually depend on the Lambda
resource "aws_sqs_queue" "input" {
  # no depends_on needed
}

# Rule: depends_on should only go in one direction
# If A depends on B, B must NOT depend on A (directly or transitively)
```

🔵 Short Interview Answer

> "Terraform detects cycles using a depth-first search topological sort on the DAG. During the graph walk, each node is marked 'in progress'. If the walk encounters a node already marked 'in progress', a

> cycle is detected and the error reports the full cycle path. The most common cycle is security groups with inline ingress/egress rules referencing each other's IDs. The fix is always to separate resource creation from rule attachment: create the security groups first (empty), then use `aws_security_group_rule` resources for the cross-references — those rules depend on both groups, but the groups don't depend on each other. Other common patterns: circular `depends_on` (one `depends_on` creates an unnecessary edge that closes a cycle) and module self-references. `terraform validate` catches cycles without making any API calls."

📊 Category 10 Summary — Quick Reference Card

Topic	One-Line Summary	Interview Weight
73. Multi-account ⚠	Provider alias + assume_role per account, external_id for confused deputy	⭐⭐⭐⭐⭐
74. Multi-cloud	Practical: SaaS providers + AWS; abstractions leak at outputs	⭐⭐⭐
75. Terragrunt	DRY backend config, run-all with dependency ordering, hooks	⭐⭐⭐⭐
76. Large-scale ⚠	Monorepo < 20 stacks; hybrid/polyrepo at scale; 50-200 resources/state	⭐⭐⭐⭐⭐
77. DAG + graph ⚠	DFS cycle detection, terraform graph	dot -Tsvg, security group fix
78. terraform test	validate (syntax), fmt (style), test (native 1.6+), mock_provider (1.7+)	⭐⭐⭐⭐⭐
79. Terratest ⚠	Go test + real infra, defer Destroy, HttpGetWithRetry, t.Parallel()	⭐⭐⭐⭐⭐
80. Mocking ⚠	mock_provider, override_resource/data/module, test in milliseconds	⭐⭐⭐⭐
81. Contract testing	Breaking changes: remove/rename var/output = major version bump	⭐⭐⭐⭐
82. Output validation	can(), regex(), alltrue() for_loop, cross-output consistency checks	⭐⭐⭐
83. TF_LOG	DEBUG=HTTP calls, TF_LOG_CORE vs TF_LOG_PROVIDER, grep patterns	⭐⭐⭐⭐
84. State corruption ⚠	Causes, detect with jq, recover from S3 versions, import orphans	⭐⭐⭐⭐⭐
85. force-unlock advanced	DynamoDB scan for batch stale locks, post-partial-apply recovery	⭐⭐⭐⭐

Topic	One-Line Summary	Interview Weight
86. Plugin crashes	Goroutine stack trace → function → GitHub issue, pin version workaround	⭐⭐⭐
87. Cycle errors ⚠	DFS in-progress marking detects cycles, SG rule separation pattern	⭐⭐⭐⭐⭐

🔑 Category 10 — Critical Rules

```
Multi-account:
  One provider alias per target account (assume_role per alias)
  Always use external_id in cross-account trust policies
  Modules must declare configuration_aliases for aliased providers
  Modules receive provider aliases via providers = {} map

Testing Pyramid (bottom to top = faster to slower):
  fmt + validate    → instant, every commit
  terraform test (mock) → milliseconds, unit testing logic
  terraform test (apply) → minutes, real infrastructure
  Terratest         → minutes, real infra + connectivity checks

State Corruption Recovery:
  1. Pull current state, check if valid JSON
  2. If invalid: restore from S3 versioned backup
  3. If missing entries: terraform import orphaned resources
  4. If phantom entries: terraform state rm
  5. Always: terraform plan after recovery to verify 0 unexpected changes

Cycle Resolution:
  Most cycles = security groups with inline cross-references
  Fix = separate creation (empty SG) from rules (aws_security_group_rule)
  Rule: if A depends on B, B cannot depend on A (transitively)
  Debug: terraform graph | dot -Tsvg to visualize

Terragrunt:
  find_in_parent_folders() = walks up directory to inherit config
  dependency {} = express cross-stack dep, mock_outputs for plan
  run-all apply = applies all stacks in topological order
  Use when: 10+ root modules with duplicate backend config
```

🎯 Category 10 — Top 5 Interview Questions to Master

1. **"How do you manage Terraform across multiple AWS accounts?"** — provider aliases with assume_role, external_id, configuration_aliases in modules, providers = {} map
2. **"Walk me through your Terraform testing strategy"** — pyramid: fmt+validate (instant), native terraform test with mocks (fast unit), terraform test with apply (integration), Terratest (e2e with

connectivity)

3. **"We're hitting a cycle error. How do you debug and fix it?"** — read the error path, `terraform graph | dot -Tsvg`, identify circular attribute references, separate creation from rule attachment (SG pattern)
4. **"How do you recover from a corrupted state file?"** — jq to diagnose, S3 versioned backup restore with state push, terraform import for orphans, state rm for phantoms, plan to verify
5. **"When would you introduce Terragrunt vs stick with vanilla Terraform?"** — Terragrunt for 10+ modules with duplicate backend config or multi-stack orchestration needs; vanilla for small teams, or when using TFC (which handles orchestration)

> 🎉 **COMPLETE** — All 107 topics across 11 categories covered! Type `quiz me category 10` or `quiz me all` to test your full knowledge. Type `deeper` on any specific topic for additional detail.